AIDLC Skill Catalogue: All Skills by Factory

Nov 11, 2026 · @Meenakshi Sundaram

Catalogue at a glance

The AIDLC needs 342 skills: 33 shared by all factories, 306 across the eight xprs-* factories and 3 domain-specific (banking) skills. The count includes 20 skills added after checking the task force's 60-skill list and the cross-cutting platform. About 20 of them overlap between factories and should be built once (see Overlaps).

Group	Focus	Skills
Shared	Jira, Confluence, hand-offs, gates, audit, git, merge request, notifications, policy, cost, gateway use, rework	33
xprs-ba / bdd-forge-ba	Ideation, requirements, epics and stories	39
xprs-sp	Sprint planning, tracking, delivery metrics	23
xprs-arch	Architecture, security design, data, test strategy	35
xprs-dev	Analysis, TDD, implementation, lint, review, merge request	59
xprs-qa	Test design and execution, defects, quality gate	37
xprs-rel	Security scans, release prep, deployment	43
xprs-ops	Monitoring, incidents, problems, support	36
xprs-mod	Debt, modernisation, skill improvement	34
Domain-specific (banking)	Banking API rules, transaction integrity, compliance rules	3
Total		342

IDs and ownership are proposals. The vote in the CIO's plan decides which skills are built first.

Shared skills used by every factory

These 33 skills are built once and reused by all eight factories, so no factory writes its own copy. Type: Core means language-neutral; Adapter means one implementation per tech stack.

ID	Skill	What it does	Type
SH-01	jira-ticket-resolution	Resolves a ticket ID to RESOLVED, NOT_FOUND or USER_SUPPLIED (exists today)	Core
SH-02	jira-ticket-read	Reads fields, links, comments and attachments of a ticket within the factory allow-list	Core
SH-03	jira-ticket-update	Adds comments, labels, status transitions and links as the agent	Core
SH-04	jira-ticket-create	Creates epics, stories, tasks, bugs and sub-tasks from approved content	Core
SH-05	confluence-page-read	Reads linked pages, specs and standards	Core
SH-06	confluence-page-publish	Publishes or updates a page from a template	Core
SH-07	repo-context-discovery	Detects language, build tool, test framework, modules and conventions of the repository	Adapter
SH-08	application-profile-load	Loads the application profile: stack, standards, owners, gates, tooling overrides	Core
SH-09	artifact-handoff-write	Writes an output artifact in its schema and registers it for the next agent	Core
SH-10	artifact-handoff-read	Reads and validates an input artifact against its schema	Core
SH-11	human-gate-request	Pauses the run, presents the evidence and records approve, reject or change	Core
SH-12	clarification-question-builder	Turns gaps into a short list of closed questions for the human	Core

ID	Skill	What it does	Type
SH-13	assumption-and-risk-log	Records each assumption and risk with owner and a way to confirm it	Core
SH-14	traceability-link	Links requirement, story, design, code, test and release evidence by ID	Core
SH-15	git-branch-and-commit	Creates the branch and commits with the ticket ID in the agreed message format	Core
SH-16	merge-request-create	Opens a merge request with description, evidence and reviewers	Core
SH-17	build-and-test-run	Runs build and unit tests through the stack adapter and returns a normalised result	Adapter
SH-18	run-summary-report	Produces the end-of-run summary: what was done, evidence, open items	Core
SH-19	audit-trail-emit	Writes the run ID, agent, action and gate decisions to the audit sink (no payload)	Core
SH-20	guardrail-escalation-handler	Handles an escalate or block verdict from the guardrails and routes it to a human	Core
SH-21	secret-and-pii-handling	Rules for what may enter a prompt, a ticket or a log; redaction before output	Core
SH-22	context-budget-management	Summarises and trims context so long runs stay inside the model window	Core
SH-23	skill-feedback-capture	Records usefulness, corrections and time saved for the vote and iteration cycle	Core
SH-24	local-model-prompt-style	Prompt conventions that suit the local Qwen model (short steps, explicit output format)	Core
SH-25	guideline-curation	Manages and updates the agent and factory standard documents and guidelines	Core
SH-26	notification-dispatch	Sends status updates to Slack, Teams or email (team's list names Slack)	Adapter
SH-27	security-policy-enforcement	Verifies at run time that actions follow security rules, using the guardrail verdicts	Core

ID	Skill	What it does	Type
SH-28	cost-metering-audit	Tracks and reports model token and infrastructure cost per run, agent and application	Core
SH-29	skill-switch-selection	Chooses and loads the right skills and adapter for the phase and tech stack at run time	Core
SH-30	mcp-tool-access-request	Requests a tool or MCP scope through the gateway and handles a denial by asking a human or choosing another route	Core
SH-31	llm-gateway-call-handling	Handles gateway outcomes: blocked, redacted, throttled or model unavailable, and retries within limits	Core
SH-32	observability-signal-emit	Emits run traces, step timings and outcome metrics to the observability service (audit events stay in SH-19)	Core
SH-33	backward-flow-rework-request	Sends found defects, rollbacks or requested changes back to the owning factory as a bounded rework item, with a rework counter	Core

xprs-ba / bdd-forge-ba: business analysis (Ideation, Requirements)

xprs-ba / bdd-forge-ba has 39 skills (later sections use the short name xprs-ba) across ideation, requirements engineering and backlog preparation. All are Core: they work on documents and tickets, not on a tech stack.

ID	Skill	What it does	Phase
BA-01	idea-intake-capture	Captures a raw idea with sponsor, problem and expected outcome	Ideation
BA-02	problem-statement-writer	Writes a one-paragraph problem statement and success measure	Ideation
BA-03	stakeholder-identification	Lists stakeholders, roles, interest and influence	Ideation

ID	Skill	What it does	Phase
BA-04	business-case-draft	Drafts benefits, cost range, risks and alternatives	Ideation
BA-05	feasibility-screening	Screens an idea for technical, data, legal and capacity feasibility	Ideation
BA-06	idea-prioritisation-scoring	Scores ideas on value, effort, risk and strategic fit	Ideation
BA-07	existing-solution-check	Searches the portfolio for an existing application or service that already does this	Ideation
BA-08	product-vision-and-scope	Writes vision, in-scope, out-of-scope and boundaries	Ideation
BA-09	stakeholder-interview-guide	Prepares questions for each stakeholder group	Requirements
BA-10	interview-notes-structuring	Turns raw meeting notes into structured findings	Requirements
BA-11	workshop-facilitation-pack	Builds the agenda, exercises and outputs for a requirements workshop	Requirements
BA-12	source-document-ingestion	Reads emails, decks and specs and extracts requirement candidates	Requirements
BA-13	business-process-as-is-mapping	Describes the current process step by step	Requirements
BA-14	business-process-to-be-mapping	Describes the target process and what changes	Requirements
BA-15	user-persona-and-journey	Writes personas and journeys for the affected users	Requirements
BA-16	functional-requirement-writer	Writes atomic, testable functional requirements with IDs	Requirements
BA-17	non-functional-requirement-writer	Writes performance, availability, security, accessibility and audit needs with numbers	Requirements
BA-18	business-rule-extraction	Extracts rules and decision tables from text	Requirements

ID	Skill	What it does	Phase
BA-19	data-requirement-capture	Lists entities, fields, sources, retention and data classification	Requirements
BA-20	integration-requirement-capture	Lists systems, interfaces, direction, volume and owners	Requirements
BA-21	regulatory-and-compliance-mapping	Maps requirements to policies and regulations that apply	Requirements
BA-22	user-story-writer	Converts requirements to stories in the agreed format	Requirements
BA-23	acceptance-criteria-writer	Writes Given/When/Then criteria for each story	Requirements
BA-24	edge-case-and-negative-scenario	Adds error, boundary and abuse scenarios to each story	Requirements
BA-25	requirement-ambiguity-check	Flags vague words, conflicts and missing values	Requirements
BA-26	requirement-completeness-check	Checks coverage against the requirement template	Requirements
BA-27	requirement-conflict-detection	Finds contradictions between requirements and across applications	Requirements
BA-28	requirement-glossary-builder	Builds the domain glossary and keeps terms consistent	Requirements
BA-29	requirement-document-compose	Assembles the full requirement document from approved parts	Requirements
BA-30	requirement-review-pack	Prepares the review pack for the product owner gate	Requirements
BA-31	requirement-change-impact	Assesses the impact of a changed requirement on scope and tests	Requirements
BA-32	epic-creation	Creates the Jira epic with goal, scope and links	Requirements
BA-33	story-breakdown-and-slicing	Slices epics into small, independently deliverable stories	Requirements
BA-34	story-estimation-support	Proposes size from similar past stories; the team confirms	Requirements

ID	Skill	What it does	Phase
BA-35	definition-of-ready-check	Verifies a story meets the team's definition of ready	Requirements
BA-36	dependency-and-risk-mapping	Records dependencies between stories, teams and systems	Requirements
BA-37	backlog-hygiene	Finds duplicates, stale items and missing fields in the backlog	Requirements
BA-38	stakeholder-signoff-record	Records who approved what and when, linked to the version	Requirements
BA-39	wireframe-translation	Converts wireframes and visual designs into requirements and acceptance criteria	Requirements

xprs-sp: sprint planning and delivery management

xprs-sp has 23 skills for sprint planning, tracking and delivery reporting. I read xprs-sp as sprint planning from the factory diagram; please confirm its scope. All are Core.

ID	Skill	What it does	Phase
SP-01	sprint-goal-drafting	Proposes a sprint goal from the top of the backlog	Requirements to Analysis
SP-02	capacity-calculation	Calculates team capacity from calendar, leave and ceremonies	Analysis
SP-03	velocity-trend-analysis	Reads past sprints to give a realistic commitment range	Analysis
SP-04	sprint-backlog-selection	Proposes the story set that fits goal and capacity	Analysis
SP-05	dependency-sequencing	Orders stories by technical and team dependencies	Analysis
SP-06	cross-team-dependency-tracking	Tracks work that waits on another team or vendor	Analysis
SP-07	task-breakdown-assignment	Splits stories into tasks and suggests owners by skill	Analysis

ID	Skill	What it does	Phase
SP-08	release-train-alignment	Aligns sprint content to the release calendar and freeze dates	Analysis
SP-09	risk-and-blocker-register	Maintains risks, blockers and mitigation owners	Development
SP-10	daily-progress-digest	Summarises ticket movement since yesterday for the stand-up	Development
SP-11	blocked-item-chaser	Finds stale or blocked tickets and drafts the nudge	Development
SP-12	scope-change-assessment	Assesses mid-sprint additions and the trade-off	Development
SP-13	burndown-and-forecast	Produces burndown and a completion forecast	Development
SP-14	work-in-progress-limit-check	Flags too many items in progress per person or column	Development
SP-15	sprint-review-pack	Builds the demo list and the done-versus-planned summary	Release
SP-16	retrospective-data-pack	Gathers facts for the retrospective: carry-over, defects, cycle time	Continuous improvement
SP-17	retrospective-action-tracking	Tracks retrospective actions to closure	Continuous improvement
SP-18	delivery-metrics-dashboard	Reports lead time, cycle time, throughput and escaped defects	Continuous improvement
SP-19	dora-metrics-collection	Collects deployment frequency, lead time, change failure rate and restore time	Continuous improvement
SP-20	stakeholder-status-report	Writes the weekly status for sponsors in plain language	All
SP-21	release-note-draft	Drafts release notes from completed tickets	Release
SP-22	ai-productivity-measurement	Measures time saved by agents against a baseline for the CIO's value case	Continuous improvement

ID	Skill	What it does	Phase
SP-23	sprint-risk-assessment	Identifies technical and delivery risks for each sprint before it starts	Analysis

xprs-arch: architecture, security design and test strategy

xprs-arch has 35 skills for solution design, security by design, data and integration design, and test strategy. Skills marked Adapter need one implementation per stack.

ID	Skill	What it does	Type
AR-01	requirement-to-architecture-analysis	Extracts architecturally significant requirements and quality attributes	Core
AR-02	architecture-option-generation	Produces two or three candidate designs with trade-offs	Core
AR-03	architecture-decision-record	Writes an ADR: context, decision, alternatives, consequences	Core
AR-04	enterprise-standards-check	Checks the design against approved patterns, technologies and principles	Core
AR-05	reference-architecture-selection	Picks the matching reference architecture or blueprint	Core
AR-06	c4-context-diagram	Draws the system context diagram	Core
AR-07	c4-container-component-diagram	Draws containers and components with responsibilities	Core
AR-08	sequence-and-flow-diagram	Draws key interaction flows and failure paths	Core
AR-09	deployment-topology-design	Designs environments, network zones, scaling and failover	Core
AR-10	api-contract-design	Designs REST or event contracts in OpenAPI or AsyncAPI	Adapter
AR-11	api-versioning-and-compatibility	Defines versioning and backward-compatibility rules	Core

ID	Skill	What it does	Type
AR-12	event-and-messaging-design	Designs topics, schemas, ordering, retry and dead letters	Core
AR-13	data-model-design	Designs entities, relations, keys and indexes	Adapter
AR-14	data-migration-design	Designs migration steps, rollback and reconciliation	Core
AR-15	data-classification-and-retention	Assigns classification, retention and masking per field	Core
AR-16	integration-pattern-selection	Chooses sync, async, batch or streaming for each interface	Core
AR-17	threat-modelling	Builds a threat model (STRIDE) with data flows and trust boundaries	Core
AR-18	security-requirement-derivation	Turns threats into concrete security requirements	Core
AR-19	authn-authz-design	Designs identity, roles, scopes and token handling	Core
AR-20	secrets-and-key-management-design	Designs how secrets are stored, rotated and injected	Core
AR-21	privacy-impact-assessment	Assesses personal data use and legal basis	Core
AR-22	resilience-design	Designs timeouts, retries, circuit breakers and graceful degradation	Core
AR-23	performance-and-capacity-design	Estimates load, sizing and performance budgets	Core
AR-24	observability-design	Defines logs, metrics, traces, alerts and dashboards for the design	Core
AR-25	cost-estimation	Estimates run and build cost of each option	Core
AR-26	build-versus-buy-assessment	Compares build, buy and reuse	Core
AR-27	technology-selection-record	Records technology choices against the approved catalogue	Core

ID	Skill	What it does	Type
AR-28	design-document-compose	Assembles the technical design document from approved parts	Core
AR-29	design-review-pack	Prepares the architecture board pack with open risks	Core
AR-30	design-to-task-mapping	Maps design components to implementation tasks for xprs-dev	Core
AR-31	test-strategy-design	Defines test levels, coverage targets, data and environments	Core
AR-32	non-functional-test-plan	Plans performance, security, resilience and accessibility tests	Core
AR-33	testability-review	Checks that the design can be tested and mocked	Core
AR-34	architecture-conformance-check	Compares built code and structure to the approved design	Adapter
AR-35	service-dependency-call-chain-mapping	Maps service-to-service call chains and what breaks if one fails	Core

xprs-dev: analysis, TDD, implementation, review and merge request

xprs-dev has 59 skills. xprs-dev-analysis, xprs-dev-technical-task-planning and the completion stage exist today and keep their names. Adapter skills need one implementation each for Java/Maven/Gradle, Angular/TypeScript, .NET, Go, Python and others.

ID	Skill	What it does	Type
DV-01	xprs-dev-analysis	Analyses the ticket and codebase impact (exists today)	Core
DV-02	codebase-impact-analysis	Finds the modules, classes and tests a change touches	Adapter
DV-03	dependency-graph-analysis	Maps internal and third-party dependencies of the change area	Adapter

ID	Skill	What it does	Type
DV-04	existing-pattern-discovery	Finds how similar features are already built here, to copy the style	Adapter
DV-05	reusable-component-search	Searches internal libraries and services before writing new code	Core
DV-06	api-and-contract-impact	Checks the change against consumers and published contracts	Core
DV-07	xprs-dev-technical-task-planning	Plans ordered technical tasks (exists today)	Core
DV-08	task-estimation-and-risk	Estimates effort and flags risky tasks	Core
DV-09	feature-branch-setup	Creates the branch with the naming convention	Core
DV-10	local-environment-bootstrap	Prepares dependencies, config and local services to run tests	Adapter
DV-11	baseline-build-and-test	Runs build and tests before any change, to know the starting state	Adapter
DV-12	test-case-derivation	Derives test cases from acceptance criteria and edge cases	Core
DV-13	failing-test-writer	Writes the failing unit test first (red)	Adapter
DV-14	test-data-and-fixture-builder	Builds fixtures, builders and anonymised data	Adapter
DV-15	mock-and-stub-generation	Generates mocks for external systems and dependencies	Adapter
DV-16	red-state-verification	Proves the new test fails for the right reason	Adapter
DV-17	minimal-implementation	Writes the smallest code that makes the test pass (green)	Adapter
DV-18	refactor-with-green-tests	Cleans up while keeping tests green	Adapter
DV-19	tdd-cycle-orchestration	Controls red, green, refactor per task and blocks skipping	Core

ID	Skill	What it does	Type
DV-20	api-endpoint-implementation	Implements controllers, handlers and routes to the contract	Adapter
DV-21	service-and-domain-logic-implementation	Implements business logic with the approved rules	Adapter
DV-22	persistence-implementation	Implements repositories, queries and entity mapping	Adapter
DV-23	database-migration-script	Writes versioned schema migrations with rollback	Adapter
DV-24	ui-component-implementation	Builds UI components, forms and state handling	Adapter
DV-25	ui-accessibility-implementation	Applies accessibility rules to UI code	Adapter
DV-26	integration-client-implementation	Implements clients for other systems with retries and timeouts	Adapter
DV-27	messaging-producer-consumer-implementation	Implements event publishers and consumers	Adapter
DV-28	configuration-and-feature-flag	Adds externalised config and feature flags	Adapter
DV-29	error-handling-and-validation	Adds input validation and consistent error responses	Adapter
DV-30	logging-metrics-tracing-instrumentation	Adds structured logs, metrics and trace context	Adapter
DV-31	secure-coding-application	Applies input handling, output encoding, authz checks and safe crypto use	Adapter
DV-32	dependency-addition-check	Checks a new library for licence, version, vulnerability and approval	Adapter
DV-33	idempotency-and-concurrency-handling	Adds idempotency keys, locking and safe retries	Adapter
DV-34	unit-test-completion	Fills missing unit tests for changed code	Adapter
DV-35	integration-test-writer	Writes tests against real or containerised dependencies	Adapter

ID	Skill	What it does	Type
DV-36	contract-test-writer	Writes consumer or provider contract tests	Adapter
DV-37	coverage-measurement	Measures line and branch coverage on changed code	Adapter
DV-38	coverage-gap-closure	Writes tests for uncovered branches until the target is met	Adapter
DV-39	mutation-test-run	Runs mutation testing (optional gate)	Adapter
DV-40	mutation-survivor-analysis	Explains surviving mutants and adds tests to kill them	Adapter
DV-41	flaky-test-detection	Reruns suspect tests and flags unstable ones	Adapter
DV-42	lint-and-format-run	Runs the linter and formatter from the stack adapter	Adapter
DV-43	lint-fix-loop	Fixes lint findings in a bounded loop, then asks for help	Adapter
DV-44	static-analysis-local	Runs local static analysis before the pipeline scans	Adapter
DV-45	code-self-review	Reviews its own diff against standards, design and acceptance criteria	Core
DV-46	design-conformance-check	Checks the code follows the approved design and ADRs	Core
DV-47	documentation-and-comment-update	Updates README, API docs and code comments	Core
DV-48	changelog-entry	Adds the changelog entry for the change	Core
DV-49	peer-review-assist	Reviews another change: bugs, style, security and test quality	Core
DV-50	review-comment-resolution	Addresses reviewer comments and re-runs checks	Core
DV-51	sonar-finding-fix	Fixes Sonar issues reported on the change	Adapter

ID	Skill	What it does	Type
DV-52	fortify-finding-fix	Fixes Fortify findings, with a human gate for false-positive claims	Adapter
DV-53	commit-message-compose	Writes a commit message from the diff and ticket	Core
DV-54	merge-request-description	Writes the MR description: change, tests, risks, evidence	Core
DV-55	merge-conflict-resolution	Resolves conflicts and re-runs tests	Adapter
DV-56	pipeline-failure-triage	Reads failed CI logs and proposes the fix	Adapter
DV-57	ticket-completion-update	Completion stage: updates the Jira ticket and links the MR (exists as a stage today)	Core
DV-58	local-run-and-smoke-check	Starts the app locally and checks the main path works	Adapter
DV-59	tdd-completeness-verification	Validates that the code satisfies every TDD requirement: a test per criterion, red seen first, all green	Core

xprs-qa: testing, defects and quality evidence

xprs-qa has 37 skills for test design, execution, defect handling and quality evidence. It also owns the Quality Defect loop.

ID	Skill	What it does	Type
QA-01	requirement-testability-review	Checks requirements and criteria can be tested	Core
QA-02	test-plan-compose	Builds the test plan from the strategy: scope, levels, entry and exit criteria	Core
QA-03	test-case-design	Designs cases using equivalence, boundary and decision-table techniques	Core

ID	Skill	What it does	Type
QA-04	test-scenario-from-acceptance	Converts acceptance criteria to test scenarios	Core
QA-05	negative-and-abuse-test-design	Designs invalid input, misuse and permission tests	Core
QA-06	test-data-design	Specifies test data needs, masking and refresh	Core
QA-07	test-data-generation	Generates synthetic data that keeps referential integrity	Adapter
QA-08	test-environment-readiness	Checks environments, versions, configs and stubs before a cycle	Core
QA-09	regression-suite-selection	Selects regression tests by code change and risk	Adapter
QA-10	test-impact-analysis	Maps changed code to the tests that must run	Adapter
QA-11	api-test-automation	Writes and runs API tests	Adapter
QA-12	ui-test-automation	Writes and runs browser or UI tests	Adapter
QA-13	mobile-test-automation	Writes and runs mobile app tests	Adapter
QA-14	integration-test-execution	Runs system integration tests across services	Adapter
QA-15	contract-test-verification	Verifies provider and consumer contracts in the pipeline	Adapter
QA-16	end-to-end-journey-test	Runs business journeys across systems	Adapter
QA-17	performance-test-design	Designs load, stress and soak profiles from the performance budget	Core
QA-18	performance-test-execution	Runs load tests and collects results	Adapter
QA-19	performance-result-analysis	Finds bottlenecks and compares with the budget and baseline	Core

ID	Skill	What it does	Type
QA-20	resilience-and-chaos-test	Tests failure handling, failover and recovery	Adapter
QA-21	accessibility-test	Runs automated and checklist accessibility checks	Adapter
QA-22	security-functional-test	Tests authentication, authorisation and input handling	Adapter
QA-23	data-migration-validation	Reconciles migrated data against the source	Adapter
QA-24	exploratory-test-charter	Writes charters for human exploratory sessions	Core
QA-25	uat-pack-preparation	Prepares UAT scripts, data and sign-off sheet for business users	Core
QA-26	test-execution-orchestration	Runs suites in order, retries safely and collects results	Adapter
QA-27	test-result-normalisation	Converts any framework's output to one result format	Adapter
QA-28	flaky-test-quarantine	Quarantines unstable tests with an owner and expiry	Adapter
QA-29	defect-logging	Creates a defect with steps, evidence, severity and links	Core
QA-30	defect-duplicate-detection	Finds existing defects that match	Core
QA-31	defect-triage-and-severity	Proposes severity, priority and owner	Core
QA-32	defect-root-cause-analysis	Traces a defect to the change or requirement that caused it	Core
QA-33	defect-reproduction	Builds a minimal reproduction from logs and steps	Adapter
QA-34	defect-fix-verification	Re-tests a fix and runs the related regression	Adapter
QA-35	quality-gate-evaluation	Evaluates coverage, mutation score, defect counts and pass rate against gate rules	Core

ID	Skill	What it does	Type
QA-36	test-summary-and-release-recommendation	Writes the test summary with evidence and a go or no-go recommendation	Core
QA-37	uat-coordination	Orchestrates user acceptance testing stages: schedules testers, collects feedback and tracks sign-off	Core

xprs-rel: security scanning, release preparation and deployment

xprs-rel has 43 skills covering DevSecOps, release preparation and deployment. Production deployment always sits behind a human gate.

ID	Skill	What it does	Type
RL-01	sast-scan-run	Runs static application security testing (Fortify or equivalent)	Adapter
RL-02	sca-dependency-scan	Scans third-party libraries for known vulnerabilities	Adapter
RL-03	licence-compliance-scan	Checks open-source licences against policy	Adapter
RL-04	secret-scan-repository	Scans code and history for committed secrets	Core
RL-05	container-image-scan	Scans images for vulnerabilities and base-image drift	Adapter
RL-06	iac-security-scan	Scans Terraform, Helm and pipeline files for misconfiguration	Adapter
RL-07	dast-scan-run	Runs dynamic scanning against a test environment	Adapter
RL-08	sonar-quality-gate-read	Reads the Sonar quality gate and issue list	Adapter
RL-09	scan-finding-normalisation	Converts findings from every tool to one severity scale	Core

ID	Skill	What it does	Type
RL-10	scan-finding-triage	Groups findings, removes duplicates, proposes false positives for human approval	Core
RL-11	vulnerability-remediation-plan	Plans fix order by severity, exposure and effort	Core
RL-12	security-exception-request	Prepares a risk acceptance request with justification and expiry	Core
RL-13	sbom-generation	Produces the software bill of materials	Adapter
RL-14	build-artifact-signing-and-provenance	Signs artifacts and records build provenance	Adapter
RL-15	pipeline-definition-generation	Creates or updates the CI/CD pipeline definition for the stack	Adapter
RL-16	build-reproducibility-check	Checks the same commit gives the same artifact	Adapter
RL-17	versioning-and-tagging	Applies semantic version and release tags	Core
RL-18	artifact-publication	Publishes build outputs to Artifactory with metadata	Adapter
RL-19	release-scope-assembly	Lists the tickets and changes in the release	Core
RL-20	release-note-compose	Writes release notes for users and operators	Core
RL-21	release-readiness-checklist	Checks tests, scans, approvals, documentation and runbooks are complete	Core
RL-22	change-request-preparation	Prepares the change record with risk, window and rollback	Core
RL-23	change-risk-assessment	Scores release risk from size, history and dependencies	Core
RL-24	deployment-plan-compose	Writes ordered deployment and verification steps	Core

ID	Skill	What it does	Type
RL-25	rollback-plan-compose	Writes and tests the rollback and data-restore steps	Core
RL-26	configuration-and-secret-promotion	Promotes environment config and secret references safely	Adapter
RL-27	database-release-coordination	Sequences schema changes with application deployment	Adapter
RL-28	environment-provisioning	Prepares environments from infrastructure as code	Adapter
RL-29	environment-drift-check	Compares target environment with the expected state	Adapter
RL-30	deployment-execution	Runs the deployment through the approved pipeline	Adapter
RL-31	canary-and-blue-green-rollout	Controls progressive rollout and traffic shift	Adapter
RL-32	feature-flag-rollout	Turns features on by cohort with monitoring	Adapter
RL-33	smoke-test-after-deploy	Runs post-deployment smoke checks	Adapter
RL-34	post-deploy-health-verification	Compares error rate, latency and saturation with the baseline	Core
RL-35	automatic-rollback-decision	Recommends rollback when health thresholds are breached	Core
RL-36	release-approval-gate-pack	Prepares the evidence pack for the production gate	Core
RL-37	deployment-record-update	Updates change record, ticket and release status	Core
RL-38	hypercare-monitoring-plan	Defines extra monitoring and owners for the first days after release	Core
RL-39	release-calendar-and-freeze-check	Checks freeze windows and conflicting releases	Core
RL-40	release-retrospective-data	Collects outcome and issues for the release review	Core

ID	Skill	What it does	Type
RL-41	ci-pipeline-trigger	Starts and monitors runs on the CI/CD server (Jenkins in the team's list)	Adapter
RL-42	pipeline-validation	Checks that CI/CD pipelines are configured correctly: stages, gates, secrets and permissions	Adapter
RL-43	rollback-readiness-check	Verifies before deployment that the rollback can really be run and data can be restored	Core

xprs-ops: monitoring, incident, problem and support

xprs-ops has 36 skills for production operations. It owns the Operational Incident loop. Any action that changes production runs only after a human gate.

ID	Skill	What it does	Type
OP-01	service-catalogue-and-runbook-lookup	Finds the service owner, dependencies and runbook	Core
OP-02	slo-and-alert-definition	Defines SLOs, error budgets and alert rules	Core
OP-03	dashboard-generation	Builds dashboards for golden signals and business metrics	Adapter
OP-04	log-query-and-search	Queries logs with the right filters and time window	Adapter
OP-05	metric-query-and-trend	Reads metrics and detects trend changes	Adapter
OP-06	distributed-trace-analysis	Follows a request across services to find the slow or failing hop	Adapter
OP-07	anomaly-detection	Flags unusual behaviour against a learned baseline	Core
OP-08	alert-noise-reduction	Groups, deduplicates and tunes noisy alerts	Core
OP-09	alert-triage	Classifies an alert, checks recent changes and decides who to call	Core

ID	Skill	What it does	Type
OP-10	incident-declaration	Opens an incident with severity, impact and commander	Core
OP-11	incident-timeline-builder	Builds a timeline from alerts, deployments and chat	Core
OP-12	impact-assessment	Estimates affected users, transactions and revenue	Core
OP-13	change-correlation	Links the incident to recent deployments and config changes	Core
OP-14	hypothesis-generation-and-testing	Lists likely causes and tests each with evidence	Core
OP-15	known-error-matching	Matches symptoms to known errors and past incidents	Core
OP-16	runbook-step-execution	Executes approved runbook steps with a gate for risky ones	Adapter
OP-17	safe-restart-and-scaling	Restarts or scales a service within pre-approved limits	Adapter
OP-18	rollback-recommendation	Recommends rollback or feature-flag off with evidence	Core
OP-19	incident-communication-draft	Drafts updates for stakeholders and status pages	Core
OP-20	escalation-routing	Routes to the right on-call team and tracks acknowledgement	Core
OP-21	incident-resolution-record	Records the fix, verification and closure criteria	Core
OP-22	post-incident-review	Writes the blameless review: cause, detection, response, actions	Core
OP-23	corrective-action-tracking	Tracks follow-up actions to closure	Core
OP-24	problem-record-creation	Opens a problem record from repeated incidents	Core
OP-25	problem-root-cause-analysis	Finds the underlying cause across incidents	Core

ID	Skill	What it does	Type
OP-26	defect-ticket-from-incident	Creates a development defect with evidence for the Quality Defect loop	Core
OP-27	support-ticket-triage	Classifies user tickets, finds duplicates and routes them	Core
OP-28	support-answer-from-knowledge	Drafts replies from the knowledge base with sources	Core
OP-29	knowledge-article-creation	Turns resolved cases into knowledge articles	Core
OP-30	user-impact-communication	Writes plain-language notices for planned work and outages	Core
OP-31	capacity-and-cost-monitoring	Tracks usage, saturation and cloud cost against budget	Adapter
OP-32	certificate-and-secret-expiry-watch	Warns before certificates, tokens and keys expire	Adapter
OP-33	patching-and-vulnerability-ops	Plans patching for hosts, runtimes and images	Adapter
OP-34	backup-and-restore-verification	Checks backups exist and restores work	Adapter
OP-35	disaster-recovery-drill	Plans and records DR exercises against RTO and RPO	Core
OP-36	production-feedback-to-backlog	Sends operational findings back to xprs-ba and xprs-mod as backlog items	Core

xprs-mod: modernisation, technical debt and continuous improvement

xprs-mod has 34 skills and owns the Modernization/Debt loop. Reverse-engineering and migration skills need one implementation per source and target stack.

ID	Skill	What it does	Type
MD-01	application-inventory-and-profiling	Profiles an application: size, languages, age, owners, usage	Core

ID	Skill	What it does	Type
MD-02	codebase-reverse-engineering	Reads legacy code and writes down what it does	Adapter
MD-03	business-rule-recovery	Extracts business rules from code, stored procedures and configs	Adapter
MD-04	legacy-documentation-generation	Generates architecture and API documentation from code	Adapter
MD-05	dependency-and-eol-assessment	Finds end-of-life runtimes, frameworks and libraries	Adapter
MD-06	technical-debt-detection	Finds duplication, complexity, dead code and design smells	Adapter
MD-07	technical-debt-quantification	Converts debt to effort and risk with a score per item	Core
MD-08	debt-prioritisation	Ranks debt by risk, change frequency and business value	Core
MD-09	debt-backlog-creation	Creates debt tickets with evidence and proposed fix	Core
MD-10	architecture-erosion-detection	Finds drift from the intended architecture	Adapter
MD-11	test-gap-analysis	Finds critical code with no tests	Adapter
MD-12	characterization-test-generation	Writes tests that capture current behaviour before changing it	Adapter
MD-13	safe-refactoring-plan	Plans small, reversible refactoring steps	Adapter
MD-14	automated-refactoring-execution	Applies refactorings with tests green at every step	Adapter
MD-15	dead-code-removal	Removes unused code after usage evidence	Adapter
MD-16	dependency-upgrade-planning	Plans library and framework upgrades in a safe order	Adapter
MD-17	dependency-upgrade-execution	Upgrades versions, fixes breakages and runs tests	Adapter
MD-18	language-and-runtime-upgrade	Moves to a newer language or runtime version	Adapter

ID	Skill	What it does	Type
MD-19	modernisation-strategy-selection	Chooses retain, rehost, replatform, refactor, replace or retire	Core
MD-20	target-architecture-proposal	Proposes the target design and migration path	Core
MD-21	strangler-slice-identification	Finds the first slice that can be moved safely	Core
MD-22	code-migration-translation	Translates code between languages or frameworks with a parity check	Adapter
MD-23	data-store-migration	Plans and verifies a move between databases	Adapter
MD-24	behaviour-parity-verification	Compares old and new system outputs on the same inputs	Adapter
MD-25	cloud-readiness-assessment	Checks twelve-factor and container readiness	Core
MD-26	containerisation	Creates container images and manifests for the application	Adapter
MD-27	performance-profiling-and-tuning	Finds hotspots and proposes fixes	Adapter
MD-28	cost-optimisation	Finds over-provisioned or idle resources	Core
MD-29	modernisation-roadmap	Builds the multi-quarter roadmap with effort and benefit	Core
MD-30	benefits-tracking	Measures value delivered against the business case	Core
MD-31	skill-usage-analytics	Reports which skills are used, by whom and with what outcome	Core
MD-32	skill-quality-review	Reviews a skill's results and feedback and proposes changes	Core
MD-33	skill-gap-identification	Finds repeated manual work that should become a skill	Core
MD-34	portfolio-standardisation-report	Compares the 400 applications on practices, tooling and skill adoption	Core

Domain-specific skills (banking)

Three skills in the task force's list encode banking rules rather than a lifecycle step, so they sit outside the factories. Any factory can call them; they belong to the application profile of banking applications.

ID	Skill	What it does	Type
DM-01	banking-api-design-rules	Applies the bank's API standards to designs and contracts	Domain
DM-02	transaction-integrity-check	Checks financial logic keeps ACID properties, idempotency and reconciliation	Domain
DM-03	compliance-rule-validation	Checks logic and requirements against regulatory requirements	Domain

Coverage check against the task force's list

Of the 59 skills I could read, 44 were already covered here and 15 were added. None were dropped. Row 46 is not visible in the photos and the list may continue after row 60: send those and I will check them too. Row 13 is read as project-discovery from a partly visible name.

#	Task force skill	Status	In this catalogue
1	story-authoring	Covered	BA-22, BA-23
2	technical-task-planning	Covered	DV-07
3	solution-blueprint-design	Covered	AR-02, AR-28
4	tdd-test-planning	Covered	DV-12
5	tdd-red-green-execution	Covered	DV-13, DV-16, DV-17, DV-19
6	tdd-completeness-verification	Added	DV-59
7	quality-gate-review	Covered	QA-35
8	traceability-ledger-maintenance	Covered	SH-14
9	architecture-decision-record	Covered	AR-03
10	sprint-risk-assessment	Added	SP-23 (SP-09 only keeps the register)

#	Task force skill	Status	In this catalogue
11	uat-coordination	Added	QA-37 (QA-25 only prepares the pack)
12	guideline-curation	Added	SH-25
13	project-discovery	Covered	BA-01, BA-07
14	feasibility-and-demand-assessment	Covered	BA-05, BA-06
15	impact-assessment	Covered	DV-02, BA-31
16	api-design	Covered	AR-10
17	datamodel-design	Covered	AR-13
18	threat-modeling	Covered	AR-17
19	code-generation	Covered	DV-17, DV-20 to DV-29
20	unit-test-generation	Covered	DV-13, DV-34
21	integration-test-generation	Covered	DV-35
22	refactoring-logic	Covered	DV-18
23	code-review	Covered	DV-45, DV-49
24	documentation-generation	Covered	DV-47
25	functional-test-execution	Covered	QA-26, QA-14
26	regression-test-execution	Covered	QA-09
27	performance-test-execution	Covered	QA-18
28	security-scan-execution	Covered	RL-01, RL-02, RL-07
29	build-orchestration	Covered	SH-17, RL-16
30	pipeline-validation	Added	RL-42
31	release-packaging	Covered	RL-18, RL-14
32	change-preparation	Covered	RL-22
33	deployment-execution	Covered	RL-30
34	rollback-readiness-check	Added	RL-43
35	observability-config	Covered	AR-24, DV-30, OP-03
36	incident-analysis	Covered	OP-11, OP-14

#	Task force skill	Status	In this catalogue
37	problem-management	Covered	OP-24, OP-25
38	remediation-execution	Covered	OP-16, OP-17
39	technical-debt-identification	Covered	MD-06
40	performance-optimization	Covered	MD-27
41	operational-feedback-analysis	Covered	OP-36
42	banking-api-design-rules	Added	DM-01
43	transaction-integrity-check	Added	DM-02
44	compliance-rule-validation	Added	DM-03
45	jira-issue-management	Covered	SH-01 to SH-04
47	sonarqube-analysis	Covered	RL-08
48	jenkins-pipeline-trigger	Added	RL-41
49	slack-notification-dispatch	Added	SH-26
50	cloud-resource-provision	Covered	RL-28
51	log-aggregation-query	Covered	OP-04
52	metric-threshold-check	Covered	OP-05, OP-02
53	user-journey-mapping	Covered	BA-15
54	wireframe-translation	Added	BA-39
55	dependency-graph-analysis	Added	AR-35 (DV-03 covers code dependencies only)
56	security-policy-enforcement	Added	SH-27
57	cost-metering-audit	Added	SH-28
58	version-control-tagging	Covered	RL-17
59	artifact-repository-push	Covered	RL-18
60	service-health-check	Covered	RL-34, RL-33

Cross-cutting platform: services versus skills

The six platform services are code that every agent call passes through, so they are not skills. A skill is advice the model may ignore; a service enforces. Agents still need a few thin skills to work with each service, and five were missing (SH-29 to SH-33).

Platform service	Built as	Skills that work with it
Guardrails	Code (built in v1: pre-LLM and post-LLM checks)	SH-20 escalation handler, SH-21 secret and PII handling, SH-27 policy enforcement
LLM Gateway	Code (proxy in front of the local Qwen model)	SH-31 gateway call handling (new), SH-24 prompt style, SH-22 context budget
MCP Gateway	Code (tool and MCP policy, allow-lists)	SH-30 tool access request (new), SH-02 to SH-04 and SH-05, SH-06 for Jira and Confluence
Context Setting	Code that loads the application profile and standards	SH-08 profile load, SH-10 artifact read, DM skills for banking applications
Skill Switch	Code that picks skills by phase and stack	SH-29 skill-switch selection (new), SH-07 repo context discovery
Audit / Observability	Code (audit sink, traces, metrics, cost)	SH-19 audit trail, SH-32 signal emit (new), SH-28 cost metering, SH-23 feedback capture
Backward flow to Forge	Workflow rule in the orchestrator	SH-33 rework request (new), SH-11 human gate request

The services themselves have no skill files. Enforcement stays in code so that no skill can skip it.

Overlaps to consolidate

Twenty pairs of skills appear in more than one factory. Each should be built once under the owner shown, and the other factory calls it. The owner is my proposal for the task force to confirm.

Overlap	Skills	Proposed owner	Other factory's role
Release notes	SP-21, RL-20	xprs-rel	xprs-sp supplies the ticket list
Defect creation	QA-29, OP-26	xprs-qa	xprs-ops passes incident evidence
Rollback decision	RL-25, RL-35, OP-18	xprs-rel for the plan, xprs-ops for live recommendation	Each reads the other's output
Post-deploy health	RL-34, OP-05	xprs-ops (metric query)	xprs-rel sets thresholds and decides
Observability design and set-up	AR-24, OP-02, OP-03	xprs-arch designs, xprs-ops implements	Shared signal definitions
Scope change impact	BA-31, SP-12	xprs-ba	xprs-sp adds sprint capacity effect
Dependencies	BA-36, SP-05, SP-06	xprs-sp	xprs-ba records requirement dependencies
Estimation	BA-34, DV-08	xprs-dev	xprs-ba asks for a size
Test strategy and plan	AR-31, QA-02	xprs-arch strategy, xprs-qa plan	Plan must trace to strategy
Performance tests	AR-32, QA-17	xprs-qa	xprs-arch supplies budgets
Design conformance	AR-34, DV-46	xprs-arch (check logic)	xprs-dev runs it before the merge request
Flaky tests	DV-41, QA-28	xprs-qa	xprs-dev detects and reports
Coverage and mutation	DV-37, DV-39, QA-35	xprs-dev runs, xprs-qa evaluates the gate	One result format (QA-27)
Scan fixing	DV-51, DV-52, RL-01, RL-08	xprs-rel scans, xprs-dev fixes	Same normalised finding format (RL-09)

Overlap	Skills	Proposed owner	Other factory's role
Local static analysis	DV-44, MD-06	xprs-dev (per change)	xprs-mod reuses it for portfolio-wide debt
Secret scanning	RL-04, SH-21	Shared	RL-04 is the scan; SH-21 is the rule set
Pipeline files	RL-15, DV-56	xprs-rel	xprs-dev triages failures
Documentation	DV-47, MD-04	xprs-dev (per change)	xprs-mod generates for legacy code
Dependency upgrades	DV-32, MD-16, MD-17	xprs-mod	xprs-dev checks new additions only
Delivery metrics and value	SP-18, SP-19, SP-22, MD-30, MD-31	xprs-sp (delivery), xprs-mod (skills and benefits)	One metrics store

Next steps: voting and the skill template

The catalogue is step 1 of the CIO's skills approach; the vote on it is step 2, and the skill template unlocks step 3 (build).

- [ ] Confirm the factory scope, especially xprs-sp (read here as sprint planning and delivery management)
- [ ] Task force confirms the owner for each overlap in the previous section
- [ ] Score each skill on business impact, frequency of use, automation potential and complexity (the criteria in the CIO's email), then vote
- [ ] Share the available skill templates so this catalogue can be aligned to them
- [ ] Pick the first build wave: a small set across factories that gives an end-to-end flow, for example the phase-1 pipeline skills in xprs-ba, xprs-dev and the shared group
- [ ] Run the OpenCode spike on hook coverage so guardrail enforcement is settled before skills are built

Working assumptions

- IDs use a factory prefix and number (SH, BA, SP, AR, DV, QA, RL, OP, MD) and can be renamed.
- Until the user's templates are shared, each skill is documented with the 8 fields already in use (description, inputs and outputs, tools and permissions, tech stack, dependencies, templates, validation, reuse and enhancement), plus the CIO's

purpose, inputs, outputs, execution, tools and quality criteria.

- Type Adapter means the skill logic is common and the commands differ per tech stack; the adapter contract covers detect, build and test, coverage, mutation, lint and scan arguments.